

http://algs4.cs.princeton.edu

3.1 SYMBOL TABLES

- *API*
- *elementary implementations*
- *ordered operations*



3.1 Symbol Tables

- *API*
- *elementary implementations*
- *ordered operations*

Symbol tables

Key-value pair abstraction.

- Insert një vlerë me çelës të specifikecifikuar.
- Le të jipet një, search për vlerë që korrespondon.

P.Sh. DNS lookup.

- Insert domain name me IP address të specifikuar.
- Le të jipet një domain name, të gjindet IP adresa që korrespondon.

domain name	IP address
www.cs.princeton.edu	128.112.136.11
www.princeton.edu	128.112.128.15
www.yale.edu	130.132.143.21
www.harvard.edu	128.103.060.55
www.simpsons.com	209.052.165.60

çelësi

vlera

Symbol table applications

aplikimi	qëllimi për search	çelësi	vlera
dictionary	find definition	word	definition
book index	find relevant pages	term	list of page numbers
file share	find song to download	name of song	computer ID
financial account	process transactions	account number	transaction details
web search	find relevant web pages	keyword	list of page names
compiler	find properties of variables	variable name	type and value
routing table	route Internet packets	destination	best route
DNS	find IP address	domain name	IP address
reverse DNS	find domain name	IP address	domain name
genomics	find markers	DNA string	known positions
file system	find file on disk	filename	location on disk

Symbol tables: konteksti

Gjithashtu njihet si: maps, fjalorë, array-at asociative.

Gjeneralizon arrays. Çelësat nuk mund të jenë në mes të 0 dhe $N-1$.

Përkrahja tek gjuhët programuese.

- Libraritë eksterne: C, VisualBasic, Standard ML, bash, ...
- Built-in libraritë: Java, C#, C++, Scala,
- Built-in në gjuhë: Awk, Perl, PHP, Tcl, JavaScript, Python, Ruby, Lua.

çdo array është një array asociative

çdo objekt është një array asociative

tabela është struktura e vetme primitive

```
hasNiceSyntaxForAssociativeArrays["Python"] = true
hasNiceSyntaxForAssociativeArrays["Java"]   = false
```

Python code i vlefshëm

API bazik për symbol table

Abstrakcioni për array asociative. Asociohet një vlerë me secilin çelës

public class ST<Key, Value>		
	ST()	*create an empty symbol table*
void	put(Key key, Value val)	*put key-value pair into the table* ← a[key] = val;
Value	get(Key key)	*value paired with key* ← a[key]
boolean	contains(Key key)	*is there a value paired with key?*
void	delete(Key key)	*remove key (and its value) from table*
boolean	isEmpty()	*is the table empty?*
int	size()	*number of key-value pairs in the table*
Iterable<Key>	keys()	*all the keys in the table*

Conventions

- Vlerat nuk janë `null`. ← Java lejon null vlera
- Metoda `get()` kthen `null` nëse çelësi nuk ekziston.
- Metoda `put()` mbishkruan vlerën e kaluar me vlerën e re.

Pasojat e implementimit.

- Implementohet lehtë `contains()`.

```
public boolean contains(Key key)
{  return get(key) != null;  }
```

- Implement version përtac (lazy version) për `delete()`.

```
public void delete(Key key)
{  put(key, null);  }
```

Çelësat dhe vlerat

Tipi i vlerës. Cilido tip gjenerik.

specify `Comparable` in API.

Tipi i çelësit: disa supozime të natyrshme.

- Supozohet se çelësat janë `Comparable`, përdoret `compareTo()`.
- Supozohet se çelësat janë cilido tip gjenerik, përdoret `equals()`.
- Supozohet se çelësat janë cilido tip gjenerik, përdoret `equals()`
  tequality; përdoret `hashCode()` për të scramble(zhurmuar) çelësin .

built-in to Java
(stay tuned)

Praktikimi. Përdoret immutable tipi për symbol table çelësa.

- Immutable në Java: `Integer`, `Double`, `String`, `java.io.File`, ...
- Mutable në Java: `StringBuilder`, `java.net.URL`, arrays, ...

Equal testimi

Të gjitha klasat në Java inherit metodën `equals()`.

Java requirements (kërkesat). Për çdo referencë `x`, `y` dhe `z`:

- Refleksive: `x.equals(x)` është `true`.
- Simetrike: `x.equals(y)` nëse `y.equals(x)`.
- Transitive: nëse `x.equals(y)` dhe `y.equals(z)`,atëherë `x.equals(z)`.

equivalence relation

- Jo-null: `x.equals(null)` është `false`.

do x and y refer to the same object?

Default implementimi. `(x == y)`

Implementimi i përpunuar. `Integer`, `Double`, `String`, `java.io.File`, ...

ST test klienti për trace

Ndërtohet ST duke asociuar vlerën i me stringun e $i^{të}$ prej inputit standard.

```
public static void main(String[] args)
{
   ST<String, Integer> st = new ST<String, Integer>();
   for (int i = 0; !StdIn.isEmpty(); i++)
   {
     String key = StdIn.readString();
     st.put(key, i);
   }
   for (String s : st.keys())
      StdOut.println(s + " " + st.get(s));
}
```

çelësi	S	E	A	R	C	H	E	X	A	M	P	L	E
vlera	0	1	2	3	4	5	6	7	8	9	10	11	12

outputi

```
A 8
C 4
E 12
H 5
L 11
M 9
P 10
R 3
S 0
X 7
```



3.1 Symbol Tables

API

elementary implementations

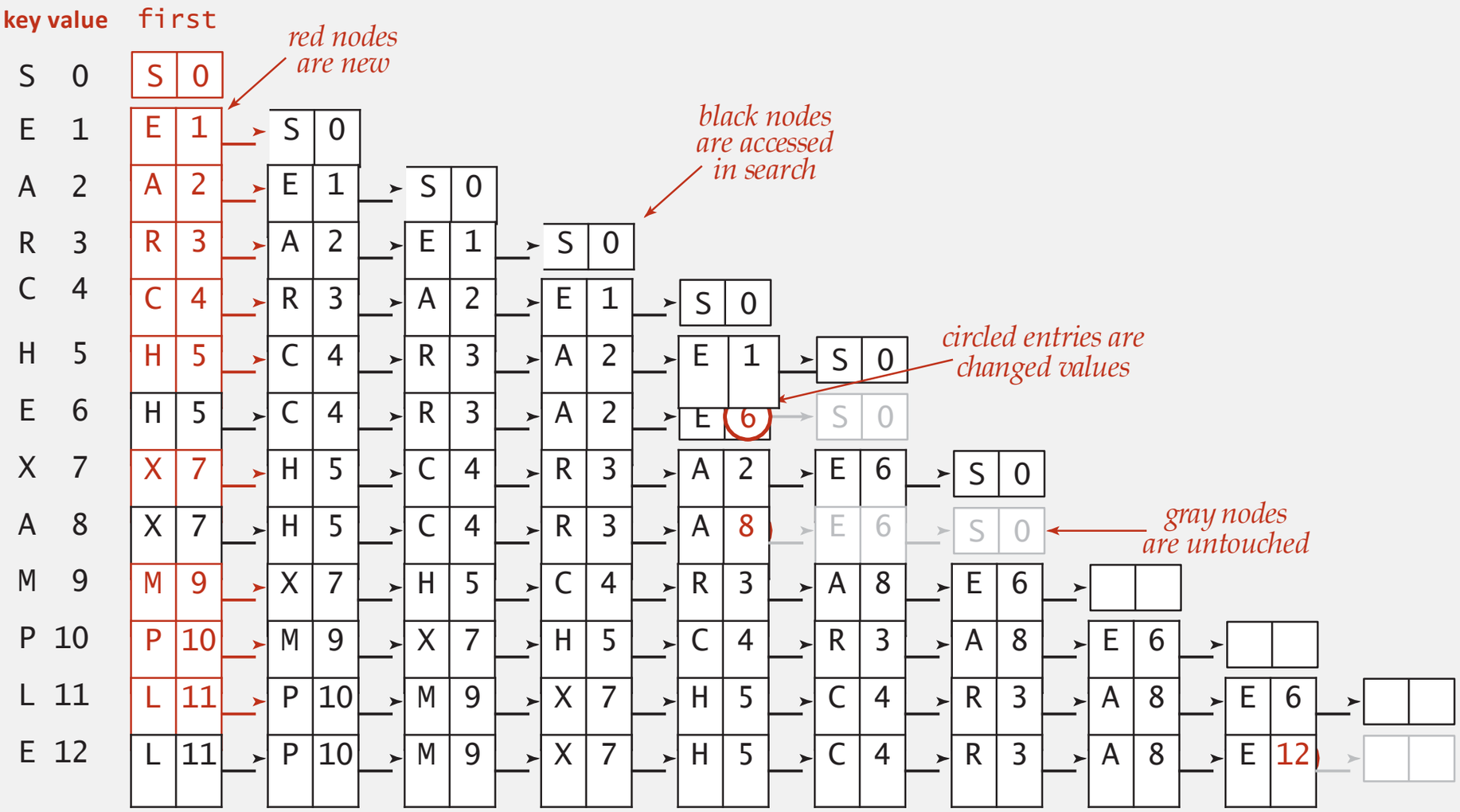
ordered operations

Kërkimi sekuencial (search) në një linked list

Struktura e të dhënave. Mbahet një linked list (e pasortuar) me key-value çifte.

Search. Skanohet nëpër të gjithë çelësat derisa të gjindet ai i përshtatshmi.

Insert. Skanohet nëpër të gjithë çelësat derisa të gjindet ai i përshtatshmi (match); nëse nuk ka match add në fillim.



Trace of linked-list ST implementation for standard indexing client

Implementimi bazik i ST: përmbledhje

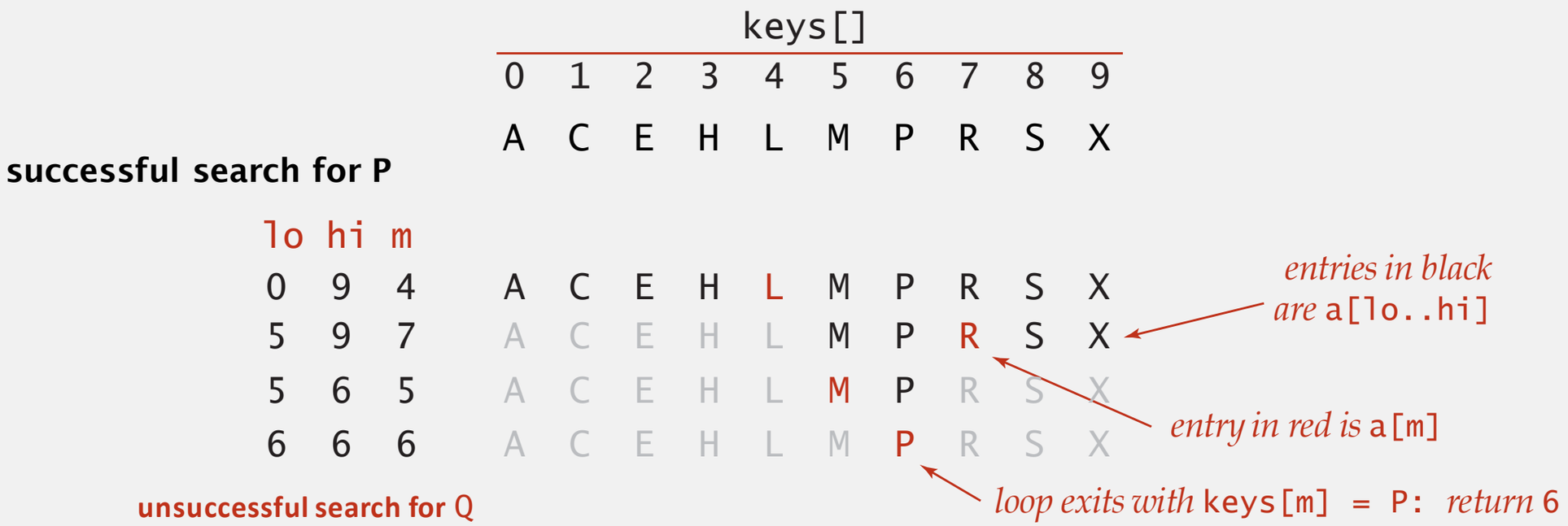
ST implementation	guarantee		average case		key interface
	search	insert	search hit	insert	
sequential search (unordered list)	N	N	$N / 2$	N	`equals()`

Sfida. Implementim efikas për search dhe insert.

Binary search në një array të sortuar

Struktura e të dhënave. Mbahet një array e sortuar e key-value (çelës-vlerë) çifteve.

Rank helper funkcioni. Sa çlësa janë $< k$?



Binary search: Java implementimi

```
public Value get(Key key)
{
   if (isEmpty()) return null;
   int i = rank(key);
   if (i < N && keys[i].compareTo(key) == 0) return vals[i];
   else return null;
}
```

```
private int rank(Key key)                          number of keys < key
{
   int lo = 0, hi = N-1;
   while (lo <= hi)
   {
       int mid = lo + (hi - lo) / 2;
       int cmp = key.compareTo(keys[mid]);
       if      (cmp  < 0) hi = mid - 1;
       else if (cmp  > 0) lo = mid + 1;
       else if (cmp == 0) return mid;
   }
   return lo;
}
```

Binary search: trace

Problemi. Për insert, duhet të shtyhen të gjithë çelësat që janë më të mëdhenjë.

key	value
S	0
E	1
A	2
R	3
C	4
H	5
E	6
X	7
A	8
M	9
P	10
L	11
E	12

keys[]

0	1	2	3	4	5	6	7	8	9
S									
E	S								
A	E	S							
A	E	R	S						
A	C	E	R	S					
A	C	E	H	R	S				
A	C	E	H	R	S	X			
A	C	E	H	M	R	S	X		
A	C	E	H	M	P	R	S	X	
A	C	E	H	L	M	P	R	S	X
A	C	E	H	L	M	P	R	S	X
A	C	E	H	L	M	P	R	S	X

entries in red were inserted

entries in gray did not move

vals[]

N	0	1	2	3	4	5	6	7	8	9
1	0									
2	1	0								
3	2	1	0							
4	2	1	3	0						
5	2	4	1	3	0					
6	2	4	1	5	3	0				
6	2	4	6	5	3	0				
7	2	4	6	5	3	0	7			
7	8	4	6	5	3	0	7			
8	8	4	6	5	9	3	0	7		
9	8	4	6	5	9	10	3	0	7	
10	8	4	6	5	11	9	10	3	0	7
10	8	4	12	5	11	9	10	3	0	7
	8	4	12	5	11	9	10	3	0	7

entries in black moved to the right

circled entries are changed values

Implementimi bazik i ST: përmbledhje

ST implementation	guarantee		average case		key interface
	search	insert	search hit	insert	
sequential search (unordered list)	N	N	$N / 2$	N	`equals()`
binary search (ordered array)	$\log N$	N	$\log N$	$N / 2$	`compareTo()`

Sfida. Implementim efikas për search dhe insert.



3.1 Symbol Tables

- *API*
- *elementary implementations*
- *ordered operations*

Shembull i një API për symbol table të sortuar



API për symbol table të sortuar

public class ST<Key extends Comparable<Key>, Value>		
Key	...	*smallest key*
	min()	
Key	max()	*largest key*
Key	floor(Key key)	*largest key less than or equal to key*
Key	ceiling(Key key)	*smallest key greater than or equal to key*
int	rank(Key key)	*number of keys less than key*
Key	select(int k)	*key of rank k*
void	deleteMin()	*delete smallest key*
void	deleteMax()	*delete largest key*
int	size(Key lo, Key hi)	*number of keys between lo and hi*
Iterable<Key>	keys()	*all keys, in sorted order*
Iterable<Key>	keys(Key lo, Key hi)	*keys between lo and hi, in sorted order*

Binary search: Operacionet me symbol table të sortuar - përmbledhje

	sequential search	binary search
search	N	$\log N$
insert / delete	N	N
min / max	N	1
floor / ceiling	N	$\log N$
rank	N	$\log N$
select	N	1
ordered iteration	$N \log N$	N

order of growth për running time të operacioneve në symbol table të sortuar